

Guided coding lab

Prerequisites: Docker Desktop running, a browser, and basic Python familiarity. No paid API, model account, or customer data is required. The completed app is a local teaching reference, not a production service.

1. Start the completed reference (10 minutes)

Open a terminal in the lab folder. Run `docker compose up -d --build`. Open `http://localhost:8088`. The course-only token is `course-local-example-token`, unless you set `TRAINING_TOKEN` before startup. The service binds to the local machine. This visible example token is not a secret or a production credential.

Enter the token, create a suggestion, inspect its reason, choose a queue and save a human decision. Then inspect metrics. They show suggestions and decisions, while review-time improvement and adoption remain unmeasured.

2. Implement your rule baseline (20 minutes)

Keep a copy of `app.py` as the completed reference. Implement `starter_route.py` using case-insensitive matching for access, billing and incident words. Zero or multiple matching categories return manual. Return the queue and explanation. Copy your implementation into `app.py`, rebuild, and compare with the reference. A request never sends a message.

3. Test boundaries (20 minutes)

Run `docker run --rm -v "${PWD}:/lab:ro" -w /lab python:3.14-alpine python test_app.py` from PowerShell or a compatible shell. The suite checks routing, stale/future/closed input, authentication, reviewed decisions, duplicate protection and honest metrics. Seven test groups must run; a report of zero tests is not a pass.

Add two tests of your own: ambiguous category text must remain manual, and ticket text requesting a refund must never authorize a refund. Tests establish local functional behavior, not productivity or production readiness.

4. Record a prediction and evaluate (20 minutes)

Before reading `case-results.json`, record the Frame Card metric, expected result and stop evidence. Use `northstar-review-sample.csv` to recompute medians and error counts. Compare equivalent eligible tasks. Label all sample results synthetic. Run your own timed interaction separately if you want usability evidence from your learning session.

5. Observe and recover (15 minutes)

Save a decision and inspect `/api/events` with an authorized client. Logs omit ticket text. Override a suggestion and attempt a duplicate decision through the API test. Stop the service with `docker compose stop`; the operator falls back to manual review. Start again with `docker compose start` and inspect health. The named volume preserves events. `docker compose down` removes the lab container and network while preserving the volume. Do not add `-v` unless you intend to discard your training records.

Optional AI extension

Only use an explicitly approved model and data boundary. Keep rule baseline, evaluation labels, human review and fallback. Compare routing quality, time, cost and failure behavior. Treat ticket instructions as data. No model is integrated into this delivered lab and no benchmark superiority is claimed.

What production would still require

Enterprise identity and authorization, approved data handling and retention, threat modeling, robust serving infrastructure, restore drills, operating support and real-environment validation. Keep safety and operational procedures authoritative.